

Tema 3: Listas e iteradores

3.1 Listas vs vectores

Las **listas** (**list**) solucionan el alto coste de inserción en medio de los vectores $\mathcal{O}(n)$. Están formadas por nodos independientes enlazados. Añadir o borrar un elemento intermedio cuesta solo $\mathcal{O}(1)$.

En algoritmia, $\mathcal{O}(n)$ (se pronuncia “O de n”) significa que **el tiempo o coste de ejecución crece de manera lineal** a medida que entran más datos. Por ejemplo: `cout << "Hello World" << endl;` es $\mathcal{O}(1)$, un elemento constante. Una peor: $\mathcal{O}(n^2)$ como `for (int i = 0; i < n; i++) { for (int j = 0; j < n; j++) { ... } }`.

Desventajas algorítmicas: - **Sin posiciones directas:** Utilizar `L[i]` genera un error de compilación. - **Coste de travesía:** Para llegar a n , es necesario recorrer secuencialmente todos los nodos anteriores.

Métodos de Listas ($\mathcal{O}(1)$ garantizado): `push_back()`, `push_front()`, `pop_back()`, `pop_front()`, `front()` y `back()`.

Aunque las listas tengan un coste constante en medio de la secuencia, en términos generales de eficiencia se acostumbra a priorizar el uso del `std::vector` dado que almacena la memoria en bloques contiguos muy rápidos de leer. Solo usaremos listas si el problema exige constantes inserciones y borrados intermedios.

3.2 Iteradores

Ante la falta de índices numéricos (como `[i]`), las listas deben recorrerse usando **Iteradores**. El iterador funciona formalmente como un puntero táctico de aquel elemento activo:

- `L.begin()`: Devuelve el iterador apuntando al **primer** elemento.
- `L.end()`: Devuelve el iterador que señala la celda virtual **después del último** elemento (fuera de rango).
- Se accede a su valor utilizando el asterisco como desreferenciación: `*it = 50`.
- Se pasa al siguiente elemento evaluando el símbolo suma: `it++`.

```
list<int> L = {10, 20, 30};
```

```
// Se usa 'auto' para simplificar tipos extremadamente largos como 'list<int>::iterator'
for (auto it = L.begin(); it != L.end(); it++) {
    *it += 5;
}
```

Variantes principales de iteradores: - **const_iterator** (`cbegin`, `cend`): Si la lista se pasa como constante `const`, no permite mutar los datos mediante `*it = x`; - **reverse_iterator** (`rbegin`, `rend`): Permite recorrer la lista del final al principio manteniendo la comodidad técnica de aplicar el `it++`.

Retroceder manualmente desde `L.end()` con iteradores conlleva problemas técnicos de índice, ya que la evaluación arranca “al límite donde ya no hay nada”. Observa cómo avanza el simulador respecto al trazado inverso:

[Simulación interactiva disponible a la web]

3.3 El peligro de alterar el itinerario avanzado: Inserciones

Borrar o añadir un elemento donde tenemos actualmente anclado el puntero en mitad de una secuencia generará prácticamente la pérdida de orientación interna, lanzando un *Segmentation Fault*: la dirección activa anterior ha quedado completamente invalidada y `it++` ya no sabe a qué objeto “siguiente” enlazar.

Por ello, en un uso de ingeniería, C++ devuelve **un nuevo iterador ya enfocado en una localización lícita** siguiente cuando usas:

- `it = L.insert(it, valor)`: Inserta **antes** de la posición y lo fija en el punto original.
- `it = L.erase(it)`: Borra el elemento y lo fija sobre el elemento a la derecha que ocupará actualmente ese hueco.

El protocolo para gestionarlo correctamente exige evitar los bucles `for` basándose en declaraciones por patrón `while`:

```
void limpiar_lista(list<int>& L) {
    auto it = L.begin();

    while (it != L.end()) {
        if (*it == 10) {
            it = L.erase(it);    // ¡Salvamos del olvido al desvincular! Devuelve el siguiente
        }
        else if (*it == -1) {
            it = L.insert(it, 0);
            advance(it, 2);      // Avanzamos el enfoque fuera del radio de lectura de la memoria
        }
        else {
            it++;                // Paso de iteración ordinaria natural
        }
    }
}
```

Este fenómeno no es único. Utilizar y recorrer con `std::vector` está sometido a los mismos efectos destructivos para el sistema si eliminas valores usando `vector.erase(it)` e intentas hacer `it++` ciegamente a continuación.

¡Visualiza paso a paso en primera persona el aseguramiento técnico del iterador observando qué roles devuelven para reenganchar al ciclo!

[Simulación interactiva disponible a la web]